

Acortador de URLs con QR

Documentación Técnica del Proyecto

José Mobarec

Marzo 2026

Índice

1. Planificación del Proyecto	3
1.1. Definición del Problema	3
1.2. Objetivos del Proyecto	3
1.3. Alcance del Sistema	4
1.4. Requerimientos Funcionales	4
1.5. Requerimientos No Funcionales	5
1.6. Tecnologías Seleccionadas	5
2. Diseño de Arquitectura del Sistema	6
2.1. Descripción General de la Arquitectura	6
2.2. Arquitectura Backend	6
2.3. Arquitectura de Base de Datos	7
2.4. Arquitectura de API REST	7
2.5. Flujo General del Sistema	8
2.6. Diagrama de Componentes	8
2.7. Diagrama de Flujo de Datos	9
3. Diseño de la Base de Datos	9
3.1. Modelo de Datos	9
3.2. Definición de Entidades	10
3.3. Relaciones entre Entidades	10
3.4. Modelo Entidad-Relación	10
3.5. Diseño de Tablas	11
3.6. Índices y Optimización	12
3.7. Estrategia de Migraciones	12

4. Mantenimiento y Mejoras Futuras	12
4.1. Escalabilidad del Sistema	12
4.2. Optimización de Rendimiento	13
4.3. Implementación de Nuevas Funcionalidades	13
4.4. Monitoreo del Sistema	14

1. Planificación del Proyecto

1.1. Definición del Problema

En el entorno digital actual, la compartición de enlaces largos puede resultar poco práctica tanto para usuarios como para sistemas que requieren integrar enlaces en espacios limitados, como redes sociales, documentos técnicos, presentaciones o códigos QR. Las URLs extensas afectan la legibilidad, dificultan la transmisión manual y pueden generar errores al momento de ser utilizadas.

Los servicios de acortamiento de URLs permiten transformar enlaces largos en identificadores cortos y fáciles de compartir. Sin embargo, muchos de estos servicios presentan limitaciones relacionadas con privacidad, dependencia de plataformas externas o falta de funcionalidades adicionales como generación automática de códigos QR o acceso a métricas de uso.

En este contexto surge la necesidad de desarrollar una solución propia que permita acortar URLs, gestionar su almacenamiento y facilitar su distribución mediante la generación automática de códigos QR. Este tipo de herramienta resulta útil tanto en entornos profesionales como en aplicaciones tecnológicas que requieren compartir enlaces de manera eficiente.

El presente proyecto propone el desarrollo de un sistema de acortamiento de URLs implementado utilizando tecnologías modernas del ecosistema de desarrollo web, específicamente Node.js para el backend y PostgreSQL como sistema gestor de base de datos. El sistema permitirá generar enlaces cortos únicos, redirigir a la URL original y generar automáticamente un código QR asociado a cada enlace.

1.2. Objetivos del Proyecto

El objetivo principal del proyecto consiste en diseñar e implementar un sistema de acortamiento de URLs con generación automática de códigos QR, empleando una arquitectura backend basada en Node.js y una base de datos PostgreSQL para el almacenamiento y gestión de la información.

De manera específica, el proyecto busca alcanzar los siguientes objetivos:

- Diseñar una arquitectura de software clara y escalable que permita gestionar la creación, almacenamiento y redirección de URLs acortadas.
- Implementar una API REST que permita la generación de URLs cortas a partir de direcciones web originales.
- Integrar un sistema automático de generación de códigos QR asociados a cada enlace acortado.

- Almacenar y gestionar la información del sistema utilizando una base de datos PostgreSQL.
- Implementar mecanismos básicos de analítica que permitan registrar el número de accesos a cada URL acortada.

1.3. Alcance del Sistema

El sistema desarrollado en este proyecto se enfocará en la implementación de un servicio backend encargado de gestionar el proceso de acortamiento de URLs, su almacenamiento y posterior redirección hacia la dirección original.

Dentro del alcance del proyecto se incluyen las siguientes funcionalidades principales:

- Generación de URLs cortas a partir de enlaces originales proporcionados por el usuario.
- Almacenamiento de la relación entre URL original y URL corta en una base de datos relacional.
- Redirección automática hacia la URL original cuando se accede a la URL corta.
- Generación automática de códigos QR asociados a cada URL acortada.
- Registro del número de accesos a cada enlace generado.
- Exposición de endpoints a través de una API REST para interactuar con el sistema.

El sistema no contempla, en su versión inicial, la implementación de una interfaz gráfica avanzada o panel administrativo. Sin embargo, la arquitectura se diseñará de manera modular para permitir la incorporación futura de nuevas funcionalidades como autenticación de usuarios, panel de administración o sistemas de analítica avanzada.

1.4. Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe proporcionar para cumplir con los objetivos establecidos.

- El sistema debe permitir registrar una URL original y generar automáticamente una URL corta única.
- El sistema debe almacenar la relación entre la URL corta y la URL original en la base de datos.
- El sistema debe redirigir automáticamente al usuario hacia la URL original cuando se acceda a la URL corta.

- El sistema debe generar un código QR asociado a cada URL corta creada.
- El sistema debe permitir consultar información básica asociada a cada enlace, como la cantidad de accesos registrados.
- El sistema debe exponer una API REST que permita la interacción con el servicio.

1.5. Requerimientos No Funcionales

Los requerimientos no funcionales establecen características relacionadas con el rendimiento, seguridad, mantenibilidad y calidad del sistema.

- El sistema debe presentar una arquitectura modular que facilite su mantenimiento y escalabilidad.
- El sistema debe implementar validación de datos para evitar el registro de URLs inválidas.
- El sistema debe utilizar variables de entorno para gestionar configuraciones sensibles.
- El sistema debe registrar errores y eventos relevantes para facilitar el monitoreo del sistema.
- El sistema debe cumplir buenas prácticas de desarrollo backend utilizadas en entornos profesionales.
- El sistema debe permitir su despliegue en entornos de contenedores utilizando Docker.

1.6. Tecnologías Seleccionadas

Para el desarrollo del proyecto se seleccionaron tecnologías ampliamente utilizadas en el desarrollo de aplicaciones backend modernas.

- **Node.js**: entorno de ejecución que permite desarrollar aplicaciones backend utilizando JavaScript, caracterizado por su modelo de ejecución basado en eventos y su alta eficiencia en aplicaciones de red.
- **Express.js**: framework minimalista para Node.js utilizado para la construcción de APIs REST y la gestión de rutas del servidor.
- **PostgreSQL**: sistema gestor de base de datos relacional robusto y altamente confiable, utilizado para almacenar la información asociada a las URLs acortadas.
- **QR Code Generator**: librería utilizada para generar códigos QR de forma dinámica a partir de las URLs acortadas.

- **Docker:** plataforma de contenedorización que permitirá empaquetar la aplicación y facilitar su despliegue en distintos entornos.
- **Git y GitHub:** herramientas de control de versiones utilizadas para gestionar el código fuente del proyecto y documentar su desarrollo como parte de un portafolio profesional.

2. Diseño de Arquitectura del Sistema

2.1. Descripción General de la Arquitectura

La arquitectura del sistema se basa en un modelo cliente-servidor, donde un backend desarrollado en Node.js se encarga de gestionar las solicitudes de los usuarios, procesar la lógica de negocio y comunicarse con una base de datos PostgreSQL para almacenar la información asociada a las URLs acortadas.

El sistema se estructura siguiendo principios de arquitectura modular, separando claramente las responsabilidades entre diferentes componentes del backend, tales como rutas, controladores, servicios y acceso a datos. Esta organización permite mejorar la mantenibilidad del código, facilitar la escalabilidad del sistema y promover buenas prácticas de desarrollo utilizadas en entornos profesionales.

El flujo general del sistema comienza cuando un usuario envía una solicitud para acortar una URL a través de la API. El backend valida la información recibida, genera un identificador corto único, almacena la relación entre la URL original y la URL corta en la base de datos y genera un código QR asociado. Posteriormente, cuando un usuario accede a la URL corta, el sistema consulta la base de datos y redirige automáticamente al destino original.

Esta arquitectura permite desacoplar la lógica del sistema y facilita futuras extensiones, como la incorporación de autenticación de usuarios, paneles de administración o sistemas avanzados de analítica.

2.2. Arquitectura Backend

El backend del sistema se implementa utilizando Node.js junto con el framework Express.js, el cual permite desarrollar una API REST de manera eficiente y estructurada.

La aplicación se organiza utilizando una arquitectura por capas que separa las distintas responsabilidades del sistema:

- **Capa de Rutas (Routes):** define los endpoints disponibles de la API y se encarga de recibir las solicitudes HTTP enviadas por los clientes.

- **Capa de Controladores (Controllers):** procesa las solicitudes recibidas desde las rutas y coordina la lógica necesaria para atender cada petición.
- **Capa de Servicios (Services):** contiene la lógica de negocio del sistema, como la generación de identificadores cortos, la creación de códigos QR y el procesamiento de información.
- **Capa de Acceso a Datos (Database):** gestiona la comunicación con la base de datos PostgreSQL y ejecuta las consultas necesarias para almacenar y recuperar información.

Esta separación permite mantener un código organizado, facilitar el mantenimiento del sistema y permitir que distintos componentes evolucionen de manera independiente.

2.3. Arquitectura de Base de Datos

El sistema utiliza PostgreSQL como sistema gestor de base de datos relacional debido a su robustez, confiabilidad y amplio soporte en aplicaciones backend modernas.

La base de datos se encarga de almacenar la información relacionada con las URLs acortadas, incluyendo la dirección original, el identificador corto generado por el sistema, la fecha de creación y el número de accesos registrados.

El modelo de datos se diseñó considerando simplicidad y eficiencia, permitiendo consultas rápidas durante el proceso de redirección. Para ello, se utilizan índices sobre los identificadores cortos, lo que optimiza el tiempo de búsqueda cuando un usuario accede a una URL acortada.

La estructura de la base de datos también permite extender el sistema en el futuro para almacenar información adicional, como estadísticas de uso, direcciones IP de acceso o metadatos asociados a cada enlace.

2.4. Arquitectura de API REST

La interacción con el sistema se realiza mediante una API REST que expone distintos endpoints para gestionar las funcionalidades del servicio de acortamiento de URLs.

El uso de una API REST permite que el sistema pueda ser utilizado por distintos tipos de clientes, como aplicaciones web, aplicaciones móviles o herramientas automatizadas.

Los principales endpoints del sistema incluyen:

- **Creación de URL corta:** permite enviar una URL original al sistema y obtener una versión acortada junto con su código QR asociado.
- **Redirección:** permite acceder a la URL corta y redirigir automáticamente al usuario hacia la URL original.

- **Consulta de estadísticas:** permite obtener información básica relacionada con el uso de cada enlace generado.

La API se diseña siguiendo convenciones RESTful, utilizando métodos HTTP apropiados, estructuras de respuesta en formato JSON y manejo adecuado de códigos de estado HTTP.

2.5. Flujo General del Sistema

El funcionamiento del sistema puede describirse mediante dos flujos principales: el flujo de creación de una URL corta y el flujo de redirección.

En el flujo de creación, el usuario envía una solicitud HTTP al endpoint correspondiente proporcionando la URL original. El servidor valida la entrada, genera un identificador corto único, almacena la información en la base de datos y genera el código QR asociado. Finalmente, el sistema devuelve la URL corta generada al cliente.

En el flujo de redirección, el usuario accede a una URL corta previamente generada. El servidor recibe la solicitud, consulta la base de datos para obtener la URL original correspondiente y redirige automáticamente al usuario hacia el destino final. Durante este proceso también se actualiza el contador de accesos asociado al enlace.

2.6. Diagrama de Componentes

El sistema está compuesto por varios componentes principales que interactúan entre sí para proporcionar las funcionalidades del servicio.

Entre los componentes más relevantes se encuentran:

- Cliente o usuario que interactúa con el sistema mediante solicitudes HTTP.
- Servidor backend desarrollado en Node.js que procesa las solicitudes y ejecuta la lógica del sistema.
- Base de datos PostgreSQL que almacena la información relacionada con las URLs acortadas.
- Módulo generador de códigos QR que permite crear representaciones visuales de los enlaces cortos.

Cada uno de estos componentes cumple una función específica dentro del sistema y se comunica con los demás a través de interfaces bien definidas.

2.7. Diagrama de Flujo de Datos

El flujo de datos del sistema describe cómo la información se mueve entre los distintos componentes durante la ejecución de las operaciones principales.

Cuando un usuario solicita la creación de una URL corta, los datos fluyen desde el cliente hacia el servidor mediante una solicitud HTTP. El servidor procesa la información, genera el identificador corto, almacena los datos en la base de datos y devuelve la respuesta al cliente.

Durante el proceso de redirección, el flujo comienza cuando un usuario accede a la URL corta. El servidor recibe la solicitud, consulta la base de datos para obtener la URL original y envía una respuesta de redirección que dirige al usuario hacia el destino correspondiente.

Este flujo de datos permite garantizar que las operaciones del sistema se ejecuten de forma eficiente y que la información se mantenga consistente en todo momento.

3. Diseño de la Base de Datos

3.1. Modelo de Datos

El diseño de la base de datos constituye un componente fundamental dentro de la arquitectura del sistema, ya que permite almacenar de manera estructurada la información asociada a las URLs acortadas y facilitar su recuperación durante el proceso de redirección.

Para este proyecto se utiliza un modelo de datos relacional implementado mediante PostgreSQL. La elección de un sistema relacional se fundamenta en su capacidad para mantener la consistencia de los datos, su alto rendimiento en consultas estructuradas y su amplia adopción en sistemas backend modernos.

El modelo de datos se diseñó considerando los siguientes principios:

- Simplicidad estructural para facilitar la comprensión y mantenimiento del sistema.
- Eficiencia en las consultas relacionadas con la redirección de URLs.
- Capacidad de extensión para incorporar futuras funcionalidades como analítica avanzada o gestión de usuarios.
- Integridad de los datos mediante restricciones y claves primarias.

El sistema se centra principalmente en almacenar la relación entre una URL original y su correspondiente identificador corto generado por el servicio.

3.2. Definición de Entidades

Una entidad representa un objeto o concepto relevante dentro del dominio del sistema que debe ser almacenado en la base de datos.

En el contexto del sistema de acortamiento de URLs, la entidad principal corresponde a los enlaces generados por el servicio. Esta entidad almacena la información necesaria para permitir la redirección hacia la URL original y registrar datos asociados a su uso.

La entidad principal del sistema es la siguiente:

- **URL:** representa una relación entre una dirección web original y su identificador corto generado por el sistema. Esta entidad contiene atributos como la URL original, el código corto, la fecha de creación y el número de accesos registrados.

La estructura de esta entidad se diseñó de forma que permita realizar consultas rápidas durante el proceso de redirección y mantener información básica asociada al uso de cada enlace generado.

3.3. Relaciones entre Entidades

En la versión inicial del sistema, el modelo de datos se compone principalmente de una entidad central que almacena la información relacionada con las URLs acortadas. Debido a la simplicidad del sistema, no se requieren relaciones complejas entre múltiples entidades.

Sin embargo, el diseño considera la posibilidad de extender el modelo en el futuro para incorporar nuevas entidades, tales como usuarios, registros de acceso o métricas detalladas de uso. En ese escenario, podrían establecerse relaciones entre la entidad de URLs y otras entidades relacionadas con autenticación o analítica.

El enfoque adoptado permite mantener un modelo de datos simple en la fase inicial del proyecto, al mismo tiempo que conserva la flexibilidad necesaria para futuras ampliaciones.

3.4. Modelo Entidad-Relación

El modelo entidad-relación describe conceptualmente cómo se organiza la información dentro de la base de datos. En este sistema, el modelo se centra en la entidad encargada de representar cada URL acortada generada por el servicio.

Cada registro dentro de esta entidad contiene la información necesaria para identificar el enlace corto, recuperar la URL original asociada y registrar información básica sobre su uso.

Los atributos principales considerados dentro del modelo incluyen:

- Identificador único del registro.

- URL original proporcionada por el usuario.
- Código corto generado por el sistema.
- Fecha de creación del enlace.
- Contador de accesos registrados.

Este modelo permite representar de forma clara la relación entre la URL corta y su destino original, garantizando al mismo tiempo la unicidad de cada identificador generado.

3.5. Diseño de Tablas

A partir del modelo conceptual definido anteriormente, se establece la estructura de las tablas que conformarán la base de datos.

La tabla principal del sistema es la encargada de almacenar las URLs acortadas. Esta tabla incluye los siguientes campos principales:

- **id**: identificador único del registro, generado automáticamente por la base de datos.
- **original_url**: dirección web original proporcionada por el usuario.
- **short_code**: identificador corto generado por el sistema que permite acceder a la URL original.
- **created_at**: fecha y hora en que se generó el enlace.
- **clicks**: contador que registra la cantidad de veces que la URL corta ha sido utilizada.

La implementación de esta estructura dentro de PostgreSQL puede representarse mediante la siguiente instrucción SQL:

```
1 CREATE TABLE urls (  
2     id SERIAL PRIMARY KEY,  
3     original_url TEXT NOT NULL,  
4     short_code VARCHAR(10) UNIQUE NOT NULL,  
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
6     clicks INTEGER DEFAULT 0  
7 );
```

Esta estructura permite almacenar de manera eficiente la relación entre la URL original y su correspondiente identificador corto, facilitando además el registro de información básica relacionada con el uso de cada enlace.

3.6. Índices y Optimización

Para garantizar un rendimiento adecuado durante el proceso de redirección, es necesario optimizar las consultas realizadas sobre la base de datos.

Dado que la operación más frecuente del sistema corresponde a la búsqueda de una URL original a partir de un código corto, se implementa un índice sobre el campo *short_code*. Este índice permite acelerar significativamente las consultas realizadas durante el proceso de redirección.

Además, el campo utilizado para almacenar el identificador corto se define como único, lo que garantiza que no existan duplicaciones dentro del sistema y permite mantener la integridad de los datos.

La combinación de índices adecuados y restricciones de unicidad contribuye a mejorar el rendimiento del sistema y a asegurar la consistencia de la información almacenada.

3.7. Estrategia de Migraciones

Las migraciones de base de datos constituyen un mecanismo que permite gestionar de forma controlada los cambios en la estructura de la base de datos a lo largo del desarrollo del proyecto.

En lugar de modificar directamente la base de datos en cada actualización, se utilizan archivos de migración que describen los cambios estructurales necesarios, como la creación de nuevas tablas, modificación de columnas o incorporación de índices.

Este enfoque permite mantener un historial de cambios en la base de datos y facilita la replicación del entorno de desarrollo en distintos sistemas o servidores.

La estrategia de migraciones adoptada en este proyecto permitirá garantizar la consistencia de la estructura de datos durante el desarrollo, las pruebas y el despliegue del sistema.

4. Mantenimiento y Mejoras Futuras

El desarrollo de una aplicación web no concluye con su implementación inicial. Para garantizar la estabilidad, escalabilidad y evolución del sistema en el tiempo, resulta fundamental considerar estrategias de mantenimiento y planificación de mejoras futuras. En esta sección se presentan posibles líneas de evolución del sistema de acortamiento de URLs, enfocadas en mejorar su rendimiento, escalabilidad y capacidad funcional.

4.1. Escalabilidad del Sistema

A medida que aumenta el número de usuarios y solicitudes, el sistema debe ser capaz de mantener un rendimiento estable sin degradar la experiencia del usuario. Para lograrlo,

se pueden implementar diversas estrategias de escalabilidad tanto a nivel de aplicación como de infraestructura.

Una de las mejoras potenciales consiste en incorporar mecanismos de balanceo de carga, permitiendo distribuir las solicitudes entre múltiples instancias del servidor. Esto facilitaría soportar mayores volúmenes de tráfico y reducir la probabilidad de sobrecarga en un único punto del sistema.

Asimismo, la adopción de contenedores mediante tecnologías como Docker permitiría estandarizar el despliegue del sistema y facilitar su escalamiento en entornos de infraestructura en la nube. Este enfoque permitiría replicar instancias del servicio de forma dinámica según la demanda.

Finalmente, la integración de sistemas de almacenamiento en caché, como Redis, permitiría optimizar el acceso a URLs frecuentemente consultadas, reduciendo la carga sobre la base de datos y mejorando los tiempos de respuesta del sistema.

4.2. Optimización de Rendimiento

El rendimiento del sistema constituye un aspecto clave en aplicaciones que manejan un alto volumen de solicitudes de redirección. Para mejorar el desempeño general del sistema, se pueden implementar diversas estrategias de optimización.

Una posible mejora consiste en optimizar las consultas realizadas a la base de datos, mediante la utilización de índices en los campos más consultados, como el código de URL acortada. Esto permitiría reducir significativamente el tiempo necesario para localizar los registros asociados a cada redirección.

Otra estrategia relevante consiste en la implementación de mecanismos de caché para almacenar temporalmente las URLs más consultadas. De esta manera, las solicitudes frecuentes podrían resolverse sin necesidad de realizar consultas repetidas a la base de datos.

Adicionalmente, se pueden implementar técnicas de monitoreo de rendimiento que permitan identificar cuellos de botella en el sistema y aplicar mejoras específicas en aquellos componentes que presenten mayor consumo de recursos.

4.3. Implementación de Nuevas Funcionalidades

El sistema actual proporciona las funcionalidades esenciales de un servicio de acortamiento de URLs; sin embargo, existen múltiples características adicionales que podrían incorporarse para enriquecer la plataforma.

Entre las mejoras posibles se encuentra la implementación de paneles de análisis que permitan a los usuarios visualizar estadísticas detalladas sobre el uso de sus enlaces acortados, tales como número de accesos, ubicación geográfica de los usuarios o dispositivos utilizados.

Otra funcionalidad potencial consiste en permitir la creación de códigos personalizados para las URLs acortadas, lo que permitiría a los usuarios generar enlaces más descriptivos y fáciles de recordar.

También podría integrarse un sistema de autenticación de usuarios, permitiendo que cada usuario gestione sus propios enlaces y estadísticas dentro de una cuenta personal.

Finalmente, la implementación de documentación interactiva mediante herramientas como Swagger permitiría facilitar la integración del servicio con otras aplicaciones y sistemas externos.

4.4. Monitoreo del Sistema

El monitoreo continuo del sistema constituye un elemento fundamental para asegurar su estabilidad y detectar posibles fallos de manera temprana. Para este propósito, pueden integrarse herramientas de monitoreo y observabilidad que permitan supervisar el comportamiento del sistema en tiempo real.

Entre las métricas que podrían registrarse se incluyen el número de solicitudes procesadas, los tiempos de respuesta del servidor, el consumo de recursos del sistema y la tasa de errores generados por la aplicación.

Asimismo, el uso de sistemas de registro estructurado (logging) permite mantener un historial detallado de eventos y errores del sistema, facilitando las tareas de diagnóstico y mantenimiento.

La integración con plataformas de monitoreo centralizado permitiría visualizar estas métricas mediante paneles de control y generar alertas automáticas ante situaciones anómalas, contribuyendo así a mejorar la confiabilidad y disponibilidad del servicio.